Query Performance Prediction Is Not Benefit Prediction: Observability for Selective Retrieval Interventions

Anonymous Author(s)

Abstract

Selective retrieval interventions, such as LLM query expansion (QE), cross-encoder reranking, and pseudo-relevance feedback (PRF), are often gated on a query performance predictor: intervene when retrieval looks like it is failing. We show why failure is the wrong target and measure by how much. Writing the decision as maximizing an intervention's expected gain given observable telemetry, we decompose an oracle selector's gain exactly into gain on failing queries, the most any policy acting only on them can recover, and gain on healthy queries, which it never sees. On TripClick and BEIR NQ, across eight intervention, corpus, and retriever settings, this failure-only ceiling is 8–57% of the oracle's gain; a pre-registered run on an untouched benchmark (ArguAna) gives 18–39% (at a per-query cost of 0.02 nDCG@10: 8–57% and 22–41%). Detection is ordinary QPP (AUC 0.62–0.70), and under BM25 its lead over harm prediction vanishes on TripClick and NQ queries the intervention changes. At matched budgets, failure, NQC, and pre-action benefit routing, up to gradient boosting and query embeddings, stay near random, and failure routing spends most of its budget on failing queries the intervention leaves unchanged. In an exploratory analysis, observing part of the intervention helps where it usually helps: on NQ, routing reranking on cross-encoder scores of the first ten candidates beats failure routing by +0.024 nDCG@10 at matched reranking fractions and free NQC routing by +0.020 at equal measured cost, but not on TripClick or ArguAna.

1 Introduction

Retrieval interventions help some queries and harm others. LLM query expansion (QE) can rewrite a query using the evidence it retrieves [13], yet it harms a measurable fraction of queries [1, 25]; cross-encoder reranking and pseudo-relevance feedback (PRF) are equally uneven. The natural response is to intervene selectively, and the natural gate is a query performance predictor (QPP): intervene when the current ranking looks like it is failing [2, 9]. That the gate's target differs from the quantity a policy needs is not new: selective-QE work predicts expansion benefit directly [6], and uplift modelling shows that targeting high-risk cases is not targeting the cases a treatment helps [3, 8]. What is missing is how much a failure gate gives up, why, and whether the telemetry a deployed system can observe, its *observability*, can do better.

Contributions. (C1) A decision framing of selective intervention under observable telemetry, with an exact decomposition of the gain no policy acting only on failing queries can capture, at zero and positive cost (Section 3). **(C2)** Its measurement for LLM QE, reranking, and PRF on three corpora, one pre-registered and untouched: the *failure-only ceiling* is 8–57% of the oracle's gain, the pre-action predictors we test do not recover the rest, and diagnostics show why. **(C3)** Routing at matched budgets and measured cost, with partial probes of the intervention as a test of whether intervention-specific evidence changes the decision.

2 Related Work

QPP and selective QE. Pre- and post-retrieval QPP (Clarity [5], WIG [28], NQC [19]) predicts query difficulty. Cronen-Townsend et al. [6] decide whether to expand by comparing the clarity of expanded and unexpanded rankings; Amati et al. [2] gate expansion on difficulty predictors; Lv and Zhai [14] predict a per-query feedback weight; Collins-Thompson [4] trades expansion reward against risk. These works evaluate the gated system in aggregate; they neither bound what a failure gate can recover nor compare gates at matched budgets.

LLM QE and adaptive retrieval. Query2doc [24], MuGI [27], and CSQE [13] expand unconditionally; generative expansions often hurt strong retrievers [25] and fail on unfamiliar or ambiguous queries [1]. Kotte [10] gates prompt-only rewriting for dense retrievers without reliable gain. Sarode [17] finds that a learned selector over candidate LLM rewrites captures little oracle headroom and transfers inconsistently; we ask whether to intervene at all, across three interventions, and bound failure-restricted policies. Adaptive-RAG [9] routes on question complexity and Mallen et al. [15] retrieve only for less popular entities; cascade rankers [23] spend costly stages only on promising candidates, and cutoff and efficiency predictors [7, 21] set per-query reranking depth or pruning. None separates a gate's detection accuracy from its benefit prediction. In uplift terms [8], our setting is unusually clean: both outcomes are observed for every query offline, so the uplift label is exact up to relevance-label noise.

3 Framework

For a query q, let R_0 be the original ranking and R_a the ranking after intervention a, with metric M (nDCG@10). The intervention's effect is $\Delta_a(q) = M(R_a, q) - M(R_0, q)$ and its cost c_a ($\Delta_0 = c_0 = 0$). With full information the best decision is $a^*(q) = \arg\max_a\{\Delta_a(q) - \kappa c_a\}$, for a cost weight κ. A deployed controller sees only telemetry Z, so the best it can do is $\pi^*(Z) = \arg\max_a\{\mathbb{E}[\Delta_a \mid Z] - \kappa c_a\}$. What matters is therefore what Z reveals about Δ_a, not about $M(R_0)$. Two information sets differ in cost: *pre-action* telemetry Z_0 describes R_0 and costs no intervention, though computing it is not free; *post-action* telemetry Z_1 also describes R_a and exists only after c_a is paid. A *probe* observes part of the intervention, such as cross-encoder scores of a few candidates, at part of its cost (Section 9). A failure gate replaces $\mathbb{E}[\Delta_a \mid Z_0]$ with $P(F \mid Z_0)$, where $F = \mathbf{1}[M(R_0, q) = 0]$.

The failure-only ceiling. Fix one intervention with $c_a = 1$, so κ is its cost per query in nDCG@10 units. The oracle selector earns $G_{\text{orc}}(\kappa) = \mathbb{E}[(\Delta - \kappa)_+]$. A policy that may act only on failing queries does best by intervening where $F = 1$ and $\Delta > \kappa$, earning $G^*_F(\kappa) = \mathbb{E}[F(\Delta - \kappa)_+]$, so

$$G_{\text{orc}}(\kappa) - G^*_F(\kappa) = \mathbb{E}[(1 - F)(\Delta - \kappa)_+]$$

(gain on healthy queries) bounds every failure-restricted policy at any $\kappa \geq 0$. Because $M \geq 0$, a failing query cannot be harmed ($\Delta \geq 0$ when $F = 1$), so at $\kappa = 0$ a *perfect* failure gate, intervening iff $F = 1$, attains G_F^*. At $\kappa > 0$ it earns $G_F(\kappa) = \mathbb{E}[F(\Delta - \kappa)]$, a further $\mathbb{E}[F(\kappa - \Delta)_+]$ below G_F^* spent on failing queries helped by less than κ, which only information beyond the binary label avoids. The identity is elementary; its use is that every term is measurable offline for any intervention. It does not bound a policy built from a failure *score* that also acts on healthy queries.

4 Setup

Data. TripClick [16]: 1.52M health documents, the TAIL bucket's validation and test splits (1,175 queries each), with click-derived relevance labels. BEIR NQ [11, 20]: 2.68M passages, human labels; 800 validation and a disjoint 2,052-query test set. BEIR ArguAna [22]: 8,674 arguments, 1,406 queries whose relevant document is the counterargument (Section 8).

Interventions. *LLM QE*: BM25 over SQLite FTS5; a deterministic rule picks one of four expansion operators or none, and `qwen2.5:3b` (4-bit, greedy, ≤ 2 calls) writes a corpus-grounded expansion. On TripClick and under dense retrieval (`bge-small-en-v1.5` [26]) the expanded ranking's top candidates are reranked by a MiniLM cross-encoder [18]; the original is not, so those labels include the reranker's effect. TripClick's deployed QE fuses original and expanded rankings (RRF); NQ's uses the expanded ranking. *Reranking*: the cross-encoder scores the BM25 top 100; its top 50 lead. *PRF*: RM3-style [12] weights from the top d documents select t terms whose BM25 ranking is interpolated with the original (weight λ); (d, t, λ) is tuned on validation over 108 settings.

Labels and models. *Failure*: $F = 1$ (45% of TripClick and 49% of NQ test queries under BM25). *Harmed*: $\Delta < 0$, ties not harmed. Detection is a logistic regression on 8 pre-action features (score shape, term coverage, rare-term IDF, entity overlap, coherence, perturbation stability; 6 under dense). Action utility is fit on Δ from post-action features (changes in score shape, coverage, and coherence, ranking overlap, rewrite drift). For pre-action routing, a *benefit* model regresses Δ on the 8 pre-action features alone. Every model and threshold is fit on validation and scored once on test; intervals are paired bootstraps over queries (10^4 resamples).

Evidence ledger. Only the ArguAna results (Section 8) are independent confirmation: every NQ query and the TripClick TAIL test set have been inspected. *Fixed before a test run*: the NQ, reranking, and PRF runs. *Pre-committed analyses of inspected traces*: the AUCs, ceiling, budget routing, and Holm correction. *Exploratory*: TripClick test was used during development, the changed-query columns of Table 2 were added after seeing the main results, and the NQ B1/B2 runs, diagnostics, and probes of Section 9 followed protocols frozen before computing but after the test sets were inspected; the NQC probe contrasts, Spearman intervals, and the positive-cost ceiling, and P95 costing were added after review. Precommitments and scripts are in the anonymized repository: https://anonymous.4open.science/r/agentic-query-expansion-8939/.

5 The Failure-Only Ceiling

A perfect failure detector would capture 8–57% of the oracle's gain (Table 1; 18–39% on ArguAna), though it never makes a query worse. The share follows how often an intervention rescues a failing query relative to how often it helps a healthy one: reranking rescues often (17% and 46% of failing queries), so half its headroom lies there; LLM QE rescues 6–15% (a third to two-fifths); PRF on TripClick almost never (0.8%). The rest sits in healthy queries that the same intervention also harms often (19–56%), which only a benefit prediction can separate.

Table 1: Failure-only ceiling (test). Rescue: $P(\Delta > 0 \mid F)$. Healthy helped / harmed: $P(\Delta \gtrless 0 \mid \neg F)$. Share: perfect failure gate's gain over the oracle's at $\kappa = 0$, 95% CI. ArguAna: pre-registered, untouched test ($n = 430$).

Setting	$P(F)$	Rescue	Healthy help	harm	Share
LLM QE, TripClick	.45	.07	.15	.19	42% [32, 52]
LLM QE, NQ	.49	.07	.19	.33	33% [26, 40]
LLM QE, TripClick, dense	.59	.06	.25	.41	34% [25, 43]
LLM QE, NQ, dense	.26	.15	.19	.29	33% [27, 40]
Reranking, TripClick	.45	.17	.24	.56	52% [44, 60]
Reranking, NQ	.49	.46	.55	.19	57% [54, 60]
PRF, TripClick	.45	.01	.12	.20	8% [1, 17]
PRF, NQ	.49	.05	.20	.22	32% [25, 39]
Reranking, ArguAna	.21	.36	.25	.53	38% [27, 49]
PRF, ArguAna	.21	.01	.05	.06	18% [0, 52]
LLM QE, ArguAna	.21	.27	.23	.59	39% [27, 52]

The share is measured at each κ, not constant: it can move either way as κ grows. At $\kappa = 0.02$ the best failure-restricted policy reaches 8–57% of the oracle's gain (22–41% on ArguAna; per setting in `docs/restricted_oracle.json`). Intervening on every failing query does worse, paying a further 0.004–0.011 on failures helped by less than κ; it loses outright for PRF on both corpora and dense TripClick QE, while the oracle still gains 0.011–0.027.

Which queries a failure gate picks. At a 25% budget (exploratory; categories prespecified, summaries chosen after review), 45–68% of the queries failure routing selects on TripClick and NQ (30–36% on ArguAna) are failing queries the intervention leaves unchanged, and rescues appear among them at about their base rate. Healthy queries gaining at least 0.10 hold 42–83% of the oracle's gain and small gains at most 9% (except ArguAna PRF), so what is missed is large gains, not noise around zero. Among the queries each pre-action scorer selects (failure, NQC, linear or boosted benefit), the share helped is within −0.11 to +0.08 of its base rate, and they capture 13–41% of the gain of the oracle's selection, against 25–35% for random.

6 Detection vs. Action Utility

Detection is ordinary QPP. Detection AUC is 0.62–0.70; NQC [19] matches it on TripClick (0.647 vs. 0.649) and beats it on NQ (0.721 vs. 0.680).

Its lead over harm prediction is structural. Detection beats harm AUC in seven of eight settings after Holm correction (Table 2), and in six of eight when harm requires $\Delta < -0.05$ or -0.10. But failing queries cannot be harmed and are mostly left unchanged, so the harm label is largely a failure indicator. On queries the intervention changed, detection drops to 0.47–0.58 and every TripClick and NQ BM25 gap vanishes; only dense NQ keeps one, and ArguAna PRF's rests on 38 queries (21 harmed). The gap survives a harm model on pre- plus post-action features (TripClick BM25 +0.040

Table 2: Detection vs. action-utility AUC on the same test queries (LLM QE: acted-upon queries). Diff.: detection minus harm AUC, paired bootstrap; † not significant after Holm over the eight, ‡ over the three pre-registered ArguAna tests. $\Delta \neq 0$: detection minus direction (helped vs. harmed) on changed queries (exploratory; secondary pre-registered for ArguAna); * CI excludes 0. CIs in the repository.

Setting	Detection	Harm	Diff.	Diff., $\Delta \neq 0$
LLM QE, TripClick	0.643	0.552	+0.091	−0.044
LLM QE, NQ	0.675	0.556	+0.119	−0.041
LLM QE, TripClick, dense	0.621	0.599	+0.022†	+0.043
LLM QE, NQ, dense	0.695	0.611	+0.084	+0.122*
Reranking, TripClick	0.649	0.575	+0.075	+0.012
Reranking, NQ	0.680	0.514	+0.166	−0.025
PRF, TripClick	0.649	0.572	+0.077	+0.001
PRF, NQ	0.680	0.501	+0.179	−0.045
Reranking, ArguAna	0.646	0.570	+0.077‡	+0.045
PRF, ArguAna	0.646	0.319	+0.327	+0.458*
LLM QE, ArguAna	0.641	0.609	+0.032‡	−0.060

[+0.003, +0.078]) and relabelling TripClick QE against the reranked original (+0.059 [+0.012, +0.107]).

7 Routing at Matched Budgets

Pre-action routing. At every budget, each scorer that decides before intervening stays close to random routing and far below the oracle (Table 3). Failure routing beats random by at most 0.006 at any budget, and falls below it at 75% on TripClick for QE and reranking, up to 50% for PRF on NQ, and at every budget for reranking on ArguAna (Figure 1). Averaged over budgets, NQC is no better than the detector, and benefit routing beats failure routing only for reranking on NQ (+0.009 [+0.003, +0.015]), where it leads at every budget. Per budget (exploratory, uncorrected) it is +0.010, +0.009, +0.016, and +0.009 at 10–75%, its interval excluding zero except at 25%; ArguAna's B1 reranking lead is +0.018 [+0.001, +0.033] at 10%, its intervals spanning zero at 25–75%. Boosted and embedding predictors (B1, B2; Section 8), fit on NQ validation (exploratory), do no better: none survives Holm, and the best recovers 8% of the oracle-minus-failure headroom.

8 Pre-Registered Replication on ArguAna

To test the conclusions on data never inspected for this work, we pre-registered a run on BEIR ArguAna, a counterargument-retrieval task, before downloading it: hypotheses, a debate-grouped split (701/275/430), the same three interventions (LLM QE as on NQ, without a reranker), three new pre-action benefit predictors, and Holm-corrected primary contrasts. Each query is itself a corpus document, and 108 arguments are cross-listed under two ids; every search drops the query's own and text-identical documents, and duplicates share a split. These fixes, and three primary contrasts adopted from an externally written protocol, were dated amendments made before any label was read (`configs/arguana_precommit.json`). Predictors are fit on training queries, selected on validation, and scored once on test. B1, B3, and the failure router use 10 pre-action features (the 8 above, NQC, query length). B1 is LightGBM regression on Δ (two depth/leaves/L2 settings, chosen by validation MSE). B2 uses the embedding alone: the inverse-distance-weighted mean Δ of the k nearest training queries under frozen `bge-small-en-v1.5` embeddings (k by validation MSE). B3 is LightGBM quantile regression; its registered quantile choice by pinball loss is unsound (losses at different quantiles are incomparable), so a secondary B3 fixes it at 0.25. Grids: `scripts/54_baseline_runner.py`.

On ArguAna every intervention loses on average (reranking −0.071 [−0.108, −0.034], LLM QE −0.163), yet the oracle could still gain 0.053 and 0.010 budget-averaged, so the question is purely one of selection. PRF barely acts here (nDCG@10 changes for 9% of queries), so its rows are uninformative. Pre-action routing again stays near random (Table 4). For reranking, B1 minus failure routing is +0.015 [−0.001, +0.028] (Holm $p = 0.13$), B2 minus failure +0.008 [−0.005, +0.020] ($p = 0.37$), and B3 versus B1 under a budget cap +0.006 [−0.011, +0.023] ($p = 0.52$); the best recovers 16% of the oracle-minus-failure headroom, below the pre-registered 25% bar. Per budget, failure routing of reranking is below random at every budget and B1 above it (Figure 1); at 10%, 42% of the queries failure routing picks are harmed. A secondary pre-registered family (refit on train plus validation, wider grids) agrees: none of nine contrasts survives Holm. ArguAna is thus a hard, out-of-domain selection problem, not proof that pre-action prediction cannot work; B1's +0.015 may be real but below what 430 queries resolve; its features (0.32 s) cost more than PRF itself (3 ms).

9 Partial Observation and Cost

A probe sits between pre- and post-action telemetry. In an exploratory analysis (protocol frozen before any probe score was computed, on test sets already inspected), we score only the first $k \in \{5, 10, 20\}$ BM25 candidates ($k = 10$ fixed as primary in the protocol) with the cross-encoder, regress the Δ of full reranking on nine probe features (score summaries, agreement with the BM25 order, NQC, query length) with B1's model and selection rule, and fully rerank the top-scoring queries, reusing the probe's scores. Each policy is charged its median measured latency (50 validation queries per corpus): full reranking takes 0.48, 1.11, and 1.26 s on NQ, TripClick, and ArguAna, a ten-candidate probe 0.05–0.13 s, and pre-action telemetry 1.58, 0.35, and 0.32 s.

On NQ, where reranking usually helps (Figure 1), Probe-10 routing beats failure routing by +0.024 [+0.017, +0.031] and B1 by +0.017 [+0.011, +0.022] over 10–75% reranked (both Holm-significant over the six primary tests), and its predictions rank Δ better (Spearman 0.25 [0.21, 0.29], against 0.02 for the failure score, 0.09 for B1, and −0.06 [−0.10, −0.02] for NQC, the best detector). Failure and B1 routing pay for NQ's pre-action telemetry, whose perturbation search spans 2.68M passages and takes longer than reranking every query, so always reranking dominates them; against free NQC (post-review, exploratory), probe routing gains +0.030 [+0.022, +0.037] at matched fraction and +0.020 [+0.012, +0.028] at equal measured cost, both Holm-significant over the six NQC contrasts; costed at P95 rather than median latency, the latter halves to +0.010 [+0.002, +0.018]. On TripClick and ArguAna, where reranking loses on average, probes do not help: no matched-fraction contrast is significant, and B1 ranks Δ at least as well (Spearman 0.08 and 0.10 vs. 0.05; differences span zero). Across

Table 3: Pre-action routing: nDCG@10 gain over the original ($\times 10^3$) when a given share of queries is intervened on, for failure (F), benefit (B), and random (R, expected gain) routing; at 100% every policy intervenes on every query. Oracle and B−F: averages over all five budgets (registered; so Oracle can fall below All), B−F with 95% CI; * significant after Holm over the six settings.

	10%			25%			50%			75%			100%		
Setting	F	B	R	F	B	R	F	B	R	F	B	R	All	Oracle	B−F (avg.)
LLM QE, TripClick	3.5	1.2	0.1	3.0	−0.8	0.2	3.0	0.8	0.3	−1.5	1.0	0.5	0.7	19.7	−1.2 [−3.6, +1.4]
LLM QE, NQ	1.2	0.8	−2.2	−2.2	−1.2	−5.4	−5.9	−2.8	−10.8	−11.6	−12.6	−16.3	−21.7	28.4	+0.6 [−2.0, +2.9]
Rerank, TripClick	0.3	0.3	−2.8	−3.1	−0.8	−6.9	−10.1	−8.6	−13.9	−24.8	−19.7	−20.8	−27.8	40.5	+1.8 [−3.7, +6.3]
Rerank, NQ	22.1	32.6	22.5	57.6	66.9	56.3	113.5	129.2	112.5	175.1	183.7	168.8	225.1	199.5	+8.8 [+3.3, +15.0]*
PRF, TripClick	−0.7	0.3	−0.1	−0.2	0.7	−0.3	0.5	−0.2	−0.6	−0.5	−1.1	−0.8	−1.1	9.6	+0.1 [−1.0, +1.0]
PRF, NQ	0.1	0.2	1.0	1.4	1.2	2.4	3.9	5.7	4.9	7.5	7.6	7.3	9.7	24.0	+0.4 [−1.7, +2.4]

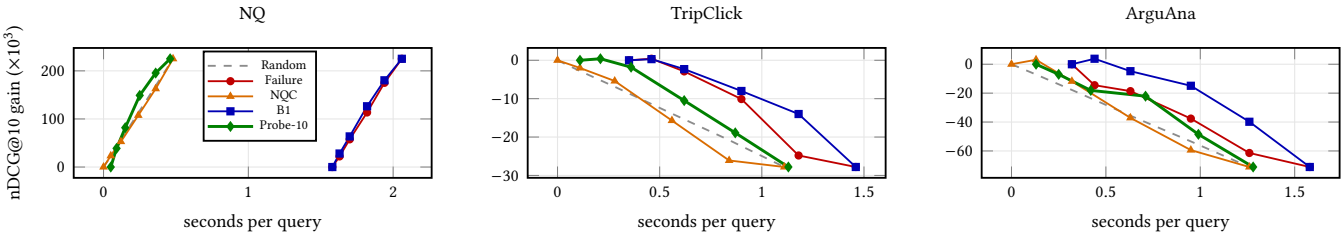


Figure 1: Reranking gain over BM25 against measured cost per query (exploratory). Points: 0, 10, 25, 50, 75, 100% of queries fully reranked; cost is pre-action telemetry (Failure, B1) or the probe, plus reranking; Probe-10's 100% point differs from full reranking by less than timing noise. The oracle (off scale) reaches 0.25, 0.06, 0.10.

Table 4: ArguAna test ($n = 430$): pre-action routing gain over BM25 ($\times 10^3$), averaged over budgets of 10–100% (registered). B1: gradient boosting on Δ; B2: k-NN over frozen query embeddings; B3: lower-quantile boosting. Always: intervening on every query (no budget); Rand. is the mean budget times Always.

	Always	Fail.	NQC	B1	B2	B3	Rand.	Oracle
Reranking	−71.1	−40.7	−35.3	−25.4	−32.2	−36.3	−37.0	52.8
PRF	−2.4	−1.2	−1.1	−1.1	−1.3	−0.4	−1.2	2.4
LLM QE	−162.8	−87.3	−91.3	−80.5	−79.7	−84.0	−84.6	9.9

sizes (secondary, uncorrected), every probe beats failure and B1 on NQ ($k = 5$: +0.020, +0.012; $k = 20$: +0.032, +0.025), none differs from either on TripClick, and on ArguAna Probe-5 trails B1 (−0.023 [−0.038, −0.006]). On TripClick the probe leads NQC at equal cost (+0.007 [+0.000, +0.013], Holm $p = 0.18$), mainly because NQC then reranks more queries (19–78% vs. 10–75%) where reranking hurts on average: equal cost rewards doing less, not selecting better.

Post-action verification, at full cost, cuts TripClick's reranking loss from −0.028 to −0.001, keeps NQ's gain (+0.223 vs. +0.225), and gains +0.004 [−0.005, +0.015] on ArguAna.

10 Discussion and Limitations

Two kinds of result show that failure is the wrong target. One is proven for failure-restricted policies: at any cost, they are capped by the oracle gain on failing queries, measured at 8–57% of the total at $\kappa = 0$ and 0.02. The other is empirical, for these representations, models, and corpora: the pre-action predictors we test pick helped queries at about their base rate, and failure routing spends most of its budget where the intervention does nothing. The second alone would fit 'no pre-action signal we test predicts benefit'; the first is what makes failure specifically the wrong target. Pre-action telemetry describes the original ranking, while Δ_a depends on what the intervention retrieves; probes, which observe some of that, help only where it usually helps (NQ). For these systems only: on NQ, always reranking is the better choice when its 0.48 s is affordable, with probes the cheaper trade-off; on TripClick and ArguAna, not reranking is better on average.

Limitations. One small (3B) generator; reranking and PRF only under BM25, with one cross-encoder and one PRF family. Three corpora, one of them (430 test queries) untouched, which leaves the ArguAna contrasts underpowered for differences below about 0.015; under dense retrieval, retriever strength and domain are confounded, and the dense QE labels include the reranker. TripClick's labels are click-derived and sparse. The cost weight κ is illustrative; latencies are single-machine medians, and probes cover reranking only. The TripClick QE benefit model is fit to the unfused validation label, since no fused validation ranking exists. Apart from ArguAna, all analyses reuse inspected test sets; every pre-committed quantity is reported in full in the repository.

11 Conclusion

Failure detection is ordinary QPP, and a controller acting only on failing queries gets at most the oracle's gain on them: 8–57% of the total at zero cost and at $\kappa = 0.02$. The pre-action predictors we test pick helped queries at about their base rate; a partial probe helps only where the intervention usually helps. Selective-intervention work should report the failure-only ceiling and cost-aware routing curves, not only AUC.

Ethical Considerations

All data (TripClick, BEIR NQ, and BEIR ArguAna) are existing research benchmarks used under their licenses; TripClick is used for non-commercial research, and no model or per-query output fit on it is released. TripClick's click logs are aggregate, de-identified interactions, and we make no attempt to re-identify searchers. No human-subjects data was collected. The generator is a small, locally hosted model used only to rewrite queries within a bounded action space. Our central finding cautions against deploying generative interventions on the strength of a failure detector alone.

References

[1] Kenya Abe, Kunihiro Takeoka, Makoto P. Kato, and Masafumi Oyamada. 2025. LLM-based Query Expansion Fails for Unfamiliar and Ambiguous Queries. *arXiv preprint arXiv:2505.12694* (2025). https://arxiv.org/abs/2505.12694

[2] Giambattista Amati, Claudio Carpineto, and Giovanni Romano. 2004. Query Difficulty, Robustness, and Selective Application of Query Expansion. In *Advances in Information Retrieval: 26th European Conference on IR Research (ECIR) (LNCS, Vol. 2997)*. 127–137. doi:10.1007/978-3-540-24752-4_10

[3] Eva Ascarza. 2018. Retention Futility: Targeting High-Risk Customers Might Be Ineffective. *Journal of Marketing Research* 55, 1 (2018), 80–98. doi:10.1509/jmr.16.0163

[4] Kevyn Collins-Thompson. 2009. Reducing the Risk of Query Expansion via Robust Constrained Optimization. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 837–846. doi:10.1145/1645953.1646059

[5] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2002. Predicting Query Performance. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 299–306. doi:10.1145/564376.564429

[6] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2004. A Framework for Selective Query Expansion. In *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management*. 236–237. doi:10.1145/1031171.1031220

[7] J. Shane Culpepper, Charles L. A. Clarke, and Jimmy Lin. 2016. Dynamic Cutoff Prediction in Multi-Stage Retrieval Systems. In *Proceedings of the 21st Australasian Document Computing Symposium (ADCS)*. 17–24.

[8] Carlos Fernández-Loría and Foster Provost. 2022. Causal Classification: Treatment Effect Estimation vs. Outcome Prediction. *Journal of Machine Learning Research* 23, 59 (2022), 1–35.

[9] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*. https://aclanthology.org/2024.naacl-long.389/

[10] Varun Kotte. 2026. Not All Queries Need Rewriting: When Prompt-Only LLM Refinement Helps and Hurts Dense Retrieval. *arXiv preprint arXiv:2603.13301* (2026). https://arxiv.org/abs/2603.13301

[11] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, et al. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl_a_00276

[12] Victor Lavrenko and W. Bruce Croft. 2001. Relevance-Based Language Models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 120–127. doi:10.1145/383952.383972

[13] Yibin Lei, Yu Cao, Tianyi Zhou, Tao Shen, and Andrew Yates. 2024. Corpus-Steered Query Expansion with Large Language Models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*. 393–401.

[14] Yuanhua Lv and ChengXiang Zhai. 2009. Adaptive Relevance Feedback in Information Retrieval. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 255–264. doi:10.1145/1645953.1645988

[15] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9802–9822. doi:10.18653/v1/2023.acl-long.546

[16] Navid Rekabsaz, Oleg Lesota, Markus Schedl, Jon Brassey, and Carsten Eickhoff. 2021. TripClick: The Log Files of a Large Health Web Search Engine. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2507–2513. doi:10.1145/3404835.3463242

[17] Gauri Sarode. 2026. Not Every Expansion Helps: The Selection and Generalization Gap in LLM Query Expansion. In *Proceedings of the 14th ACM IKDD International Conference on Data Science (CODS)*. To appear.

[18] Sentence Transformers. 2026. cross-encoder/ms-marco-MiniLM-L6-v2. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2

[19] Anna Shtok, Oren Kurland, David Carmel, Fiana Raiber, and Gad Markovits. 2012. Predicting Query Performance by Query-Drift Estimation. *ACM Transactions on Information Systems* 30, 2 (2012), 11:1–11:35. doi:10.1145/2180868.2180873

[20] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. https://arxiv.org/abs/2104.08663

[21] Nicola Tonellotto, Craig Macdonald, and Iadh Ounis. 2013. Efficient and Effective Retrieval Using Selective Pruning. In *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining (WSDM)*. doi:10.1145/2433396.2433407

[22] Henning Wachsmuth, Shahbaz Syed, and Benno Stein. 2018. Retrieval of the Best Counterargument without Prior Topic Knowledge. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 241–251.

[23] Lidan Wang, Jimmy Lin, and Donald Metzler. 2011. A Cascade Ranking Model for Efficient Ranked Retrieval. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 105–114. doi:10.1145/2009916.2009934

[24] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9414–9423. doi:10.18653/v1/2023.emnlp-main.585

[25] Orion Weller, Kyle Lo, David Wadden, Dawn Lawrie, Benjamin Van Durme, Arman Cohan, and Luca Soldaini. 2024. When do Generative Query and Document Expansions Fail? A Comprehensive Study Across Methods, Retrievers, and Datasets. In *Findings of the Association for Computational Linguistics: EACL 2024*. 1987–2003.

[26] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2023. C-Pack: Packed Resources For General Chinese Embeddings. *arXiv preprint arXiv:2309.07597* (2023). https://arxiv.org/abs/2309.07597

[27] Le Zhang, Yihong Wu, Qian Yang, and Jian-Yun Nie. 2024. Exploring the Best Practices of Query Expansion with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 1872–1883. doi:10.18653/v1/2024.findings-emnlp.103

[28] Yun Zhou and W. Bruce Croft. 2007. Query Performance Prediction in Web Search Environments. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 543–550. doi:10.1145/1277741.1277835